ELI — Learning

ELI v2.3.16

August 2026

Contents

ELI Learning — LoRA Self-Training Pipeline

`eli/learning/` — 4.3k LOC, 14 files. ELI's self-improvement-via-fine-tuning path. **Important framing:** this is a *curated, human-gated, operator-invoked* training pipeline — **not** an autonomous self-modifying loop, and it targets a **separate trainable Hugging Face base**, not the inference GGUF.

Since 2.3.7 it is driven from **Labs ▸ Training** (a four-step wizard) as well as from chat actions and the overnight scheduled task. Two long-standing blockers were removed at the same time: the human review gate now has an interface, and the target system is no longer locked to Phi-3.

Files & the pipeline

The stages, roughly in order:

File	LOC	Stage
`dataset_builder.py`	558	turn ELI's logged turns/corrections into supervised examples
`dataset_filters.py`	154	quality gates (`is_bad_response`, `row_is_reviewed`)
`export_trainable_dataset.py`	168	export reviewed rows → trainable JSONL
`merge_reviewed_datasets.py`	135	merge reviewed dataset shards
`bootstrap_phi3_base.py`	245	one-time download of the trainable HF Phi-3 base
`base_model_resolver.py`	185	locate/validate the trainable base (vs an adapter)
`training_preflight.py`	142	check peft/transformers/datasets present + target ready
`lora_trainer_guard.py`	571	`TrainerTarget` plans (paths, base_family) + guard checks

File	LOC	Stage
`lora_trainer.py`	840	device selection, QLoRA, chat-template prompts, PEFT loop
`lora_eval.py`	491	eval harness (score vs expected/forbidden, inspect adapter)
`target_registry.py`	266	operator-declared training targets, any model family
`review_queue.py`	270	the human review gate: triage, approve/edit, write trainable rows

Flow

1. **Build** (`dataset_builder.py`): mines ELI's own conversation/correction history into `SupervisedExamples`. Crucially includes `clean_text` + **`redact_text` (PII redaction)** and an exclusion set so sensitive/garbage content never becomes a training example. Writes `training/datasets/eli_supervised_v0.jsonl` + a report.
2. **Gate** (`dataset_filters.py`): `is_bad_response` and `row_is_reviewed` enforce quality + a **human review flag**.
3. **Export / merge**: reviewed rows → `*.trainable.jsonl`.
4. **Preflight** (`training_preflight.py`): refuses to proceed unless the HF training stack is installed and the target/base resolve.
5. **Train** (`lora_trainer.py`): `_dataset_report` **refuses to train** on a dataset with unreviewed rows, wrong-target rows, or bad-response rows — i.e. it will not learn from un-vetted data. Loads the base through native transformers (with a RoPE-scaling compat shim for Phi-3), trains the adapter. Two things changed in 2.3.7:
   - **Prompt format follows the base model.** `_format_example` used a hardcoded Phi-3 `<|user|>/<|assistant|>` literal. Training a Qwen or Llama checkpoint on Phi-3 turn markers teaches it tokens its chat template never uses, degrading the model instead of tuning it. The tokenizer's own `chat_template` is now the source of truth, with the literal kept as a fallback for bases without one.
   - **QLoRA.** With `bitsandbytes` present the frozen base loads in 4-bit (nf4, double quant, fp16 compute) with a paged optimiser, so a card too small to hold the fp16 weights can still train the adapter.
6. **Eval** (`lora_eval.py`): `score_response` against expected/forbidden, plus `inspect_adapter` / `inspect_eval_suite`.

Device selection (`lora_trainer._pick_device`)

The old rule was a flat `free_vram >= 10 GiB` floor written for a Phi-3 on an 8 GB card, and its failure mode was a **silent** fall-through to CPU — where a run takes days and looks exactly like a hang. It also named every accelerator "cuda", showing CUDA on a Radeon.

- `_accelerator()` reports the vendor honestly. torch routes ROCm through the `torch.cuda` API (a HIP build answers `torch.cuda.is_available()`), so one code path serves NVIDIA and AMD; the vendor is read from `torch.version.hip`. Apple (mps) and Intel (xpu) are detected and **honestly marked unable** to run this trainer rather than being silently ignored.
- `estimate_vram_gb()` sizes the requirement from the checkpoint's real size on disk, so the floor holds for a 1B and for a 35B instead of a hardcoded figure.
- A refusal states what would fix it. On a 2060 Super: *"6.37 GiB free but this run needs ~9.04 GiB … free VRAM, shorten the sequence length, or install bitsandbytes for 4-bit training."*

Paths

`eli.core.paths.learning_dir()` follows the `models_dir()` pattern: the source tree in dev so the existing `training/` layout keeps working, the **user data dir** on a packaged install where `project_root()` is a read-only mount. Run logs, guard plans and the target registry all follow it, so a frozen build no longer writes into its own installation.

Targets (`target_registry.py` + `lora_trainer_guard.py`)

A **target** pairs a base model with its dataset, adapter and output directory. Until 2.3.7 the allowlist was the literal set `{eli_phi, eli_phi_ultra}` and the guard asserted `base_family == "phi3"`, so a redistributed copy running Qwen or Llama was refused at the first gate with no way to declare a target short of editing source.

The allowlist is still an allowlist — nothing trains unless it was explicitly declared — but declaring is now a **runtime act**:

- `target_registry.create_target(name, base_model_path, …)` writes an operator target into `<data dir>/training/targets.json`. Built-ins remain, so existing installs are unaffected.
- **Family is read, not asserted.** `detect_family()` reads `model_type` from the base model's own `config.json`. The guard then checks the declared family against what is actually on disk, so a target claiming `qwen3` over a Llama checkpoint is still refused. Nothing in the registry enumerates known families, so a model type released after this build still resolves.
- `base_model_resolver.discover_base_models()` scans the model roots for **any** trainable HF directory and reports each with its family and size.
- A target that has never trained has no adapter yet. That is now a *first run*, not a fault — only a malformed adapter is an error.

The review gate (`review_queue.py`)

`lora_trainer_guard` has always required every training row to be approved by a person and scoped to the target. That contract was enforced but **unreachable**: `dataset_builder` writes candidates tagged `needs_review`, and nothing in the product could retag one. Live state before this module existed: 615 candidate rows, 0 trainable, `will_train` permanently false.

`ReviewQueue` is the queue behind the Training tab:

- **Assisted triage** pre-rejects the provably unusable — `is_bad_response` matches, duplicates, instructions under 8 characters — and *flags* rows that need a human eye (very long output, redacted paths, heavy structure).
- **Triage never approves.** Approval is a person's act; there is a test pinning this. "Approve all clean rows" is one click standing for reading a filtered list, not an auto-approve.
- Editing a row **re-triages** it: an edit can rescue a rejected row and can equally break one that passed.
- `save()` writes approved rows tagged `reviewed` and scoped to the target — exactly the shape `_validate_dataset_rows` demands — overwriting rather than appending, so a second pass does not duplicate.

On the live corpus: 615 candidates → 20 auto-rejected, 309 clean, 286 flagged for judgement; approving the clean set took `train_ready` from false to **true** for the first time.

What it is — and isn't

- It **is**: a responsible, reproducible fine-tuning pipeline with PII redaction, human review gating, preflight, and an eval harness. The dataset comes from ELI's real interactions (corrections, failures, conversations).
- It **isn't**: (a) automatic — it is driven by the operator, through the Training tab, the chat actions, or a scheduled overnight job. (b) connected to the live inference model by itself — it trains an

adapter on a Hugging Face base, while inference runs a GGUF. A trained improvement reaches the running assistant only once the adapter is merged into the base and converted to GGUF (`training/merge_and_convert.py`); the Training tab says so explicitly when a run finishes rather than implying the model has changed.

Honest assessment

- **Strong (and unusually responsible):** PII redaction, refusing to train on unreviewed/bad rows, a preflight, and an eval suite are exactly what a serious fine-tuning loop needs and what most hobby projects skip. The base-vs-adapter validation prevents the classic "trained on an adapter" mistake.
- **Weak / watch:**
  1. **Base/inference disconnect** — training improves Phi-3, but the assistant usually runs another GGUF, so the self-improvement doesn't reach the live model. The loop is only closed if you actually serve the trained model.
  2. **Manual** — "self-training" is operator-driven, not autonomous. That's safe, but it means it improves only when you run it. (Arguably the right trade-off, but worth stating plainly rather than implying autonomy.)
  3. **Heavy deps** — needs the full HF/peft/transformers stack. `peft` and `datasets` were previously only in `requirements.lock.txt`, so a plain `pip install -r requirements.txt` produced an install whose Training tab could never leave preflight; both are now in `requirements.txt`, with `bitsandbytes` added for 4-bit training (excluded on macOS, which has no wheel).
  4. **Phi-3 remains the shipped default** — the built-in targets and `bootstrap_phi3_base.py` are still Phi-3 — but it is no longer a lock. Any local HF base can be declared as a target from the Training tab, and the prompt format follows the base model's own chat template rather than Phi-3's literal turn markers.
  5. **Merge-to-GGUF is still manual.** `training/merge_and_convert.py` closes the loop but depends on `unsloth`, which is in no requirements file. Producing a usable model after a successful run is therefore still a documented manual step, not a button.